

DRL Practice Worksheet

Deep Reinforcement Learning — full question & answer set

Compiled from the Module 1 & 2 self-assessment sets and the EC-2 (Mid-Sem) & EC-3 (Comprehensive) exam papers.

All numeric answers verified by direct computation. Sutton & Barto, *Reinforcement Learning*, 2nd Ed.

Session 1 — Introduction to Reinforcement Learning

Q1 · Module 1 review

How is reinforcement learning different from supervised and unsupervised learning?

✓ Solution

Supervised learning uses *labelled* input–output pairs with *instructive* feedback (the correct answer is given); unsupervised learning finds structure in *unlabelled* data. RL has no dataset: an agent *interacts* with an environment and receives a scalar *reward* that is *evaluative* (how good the action was, not what the best action was) and *delayed*; the data is sequential and non-i.i.d. RL learns a policy that maximises cumulative reward by trial and error.

Q2 · Module 1 review

“A reward signal defines the goal of an RL problem.” Explain.

✓ Solution

The reward is the only channel by which we tell the agent what we *want*. The agent’s sole objective is to maximise expected cumulative (discounted) reward, so whatever we wish to encourage must be encoded in the reward. Design the reward wrong and the agent optimises the wrong goal.

Q3 · Module 1 review

What is the exploration–exploitation trade-off? What happens to a pure explorer and a pure exploiter?

✓ Solution

Exploit = take the action currently believed best; explore = try others to improve estimates. A pure exploiter can lock onto a sub-optimal action it happened to try first; a pure explorer never cashes in on what it learned and earns low reward. Good learning needs both.

Q4 · Module 1 review

What is a model of the environment, and how does the solution differ with and without one?

✓ Solution

A model predicts dynamics: $p(s', r \mid s, a)$. With a model you can *plan* (dynamic programming) without acting. Without one (model-free) you must *learn from sampled experience*

(Monte Carlo, TD, Q-learning).

Q5 · Module 1 review Q10

What are different ways to choose ε in ε -greedy?

✓ Solution

Fixed ε (constant exploration); decaying ε (e.g. $1/t$ — explore early, exploit later, good for stationary problems); $\varepsilon = 0$ (pure greedy, only once estimates are reliable). Larger ε favours exploration (noisy/non-stationary rewards); smaller favours exploitation.

Q6 · Session 1 slides

In the tic-tac-toe TD rule $V(S_t) \leftarrow V(S_t) + \alpha[V(S_{t+1}) - V(S_t)]$, what happens if $\alpha \rightarrow 0$, if α is reduced but never 0, and if α is constant?

✓ Solution

$\alpha \rightarrow 0$: values freeze and converge (good for a stationary opponent, no further adaptation). Reduced but never 0: values converge while still allowing late corrections (ideal for a fixed opponent). Constant α : values keep tracking recent outcomes (best against a slowly-changing, non-stationary opponent).

Q7 · Exam EC-2 Q1(a), 1.5 marks

A health wristband chooses among 3 modes hourly. (a) Why RL not supervised learning? (b) Immediate reward vs long-term value?

✓ Solution

(a) No labelled “correct mode” exists; benefit is seen only after acting, choices affect future state (battery), and feedback is evaluative/delayed — i.e. sequential decision-making. (b) **Immediate reward** is this hour’s one-step benefit; **long-term value** is the expected cumulative discounted reward from this state onward under a policy. A lower-immediate-reward mode can have higher value if it leads to better future states.

Q8 · Module 1 review Q1

Define reinforcement learning in your own words, then critically inspect your definition.

✓ Solution

RL is learning what to do — mapping states to actions — to maximise a scalar reward, by trial-and-error interaction rather than from a labelled dataset. A complete definition must include: no supervisor (only a reward); delayed feedback; actions affect future states/rewards (sequential, credit assignment); and the explore–exploit balance. Omitting the sequential/delayed aspect would fail to distinguish RL from a one-shot bandit or supervised learning.

Q9 · Module 1 review Q2

What are the elements of an RL system, and what is each for?

✓ Solution

Policy $\pi(a | s)$ — the behaviour (states→action probabilities), what we want to learn.

Reward signal — defines the goal; immediate measure of good/bad. **Value function** — long-run desirability (expected cumulative reward), for far-sighted decisions. **Model** (optional) — predicts next state/reward; enables planning. (Plus the agent and environment.)

Q10 · Module 1 review Q3

Which elements define the RL *problem* vs the *solution*, and why?

✓ Solution

The problem is set by the environment, the reward signal (goal) and dynamics — no reward, no objective. The solution is the policy, usually found via a value function (and optionally a model for planning). Reward defines success (problem); policy/value deliver it (solution). Value functions are the most vital solution element, since nearly all methods estimate values.

Q11 · Module 1 review Q4

Identify a real problem solvable with RL and define its elements.

✓ Solution

Smart thermostat. Agent: the controller. Environment: house + weather + occupants. State: temperature, time, occupancy, outdoor temp. Actions: heat / cool / nothing. Reward: +comfort – energy cost. Policy: state→action rule maximising long-run comfort per unit energy. Any sequential decision problem with delayed, evaluative feedback fits.

Q12 · Module 1 review Q11(a)

Why update $V(S_t)$ only after greedy (exploiting) moves, not after exploratory moves?

✓ Solution

We want V to estimate the value of states under the *greedy* policy we intend to follow. An exploratory move is deliberately off-policy, so the state it leads to is unrepresentative; backing up its value biases estimates toward the random behaviour. Updating only after greedy moves keeps V consistent with the policy being evaluated. (Learning from exploratory moves instead yields the ϵ -soft policy's values — Ex. 1.4.)

Q13 · S&B Ex 1.1 (self-play)

If the RL agent plays tic-tac-toe against itself, what would it learn?

✓ Solution

Both sides improve together, converging toward optimal play for both — and since tic-tac-toe is solved, optimal self-play yields draws. This is a different (stronger, minimax-like)

policy than when training against a fixed imperfect opponent, where the agent instead learns to exploit that opponent's mistakes.

Q14 · S&B Ex 1.4 (exploratory moves)

What changes if we also update values after exploratory moves, and which value set is better if we keep exploring?

✓ Solution

Backing up exploratory moves makes the values converge to those of the ε -soft policy actually followed (which sometimes moves randomly), not the greedy policy. If we keep exploring forever, the set that *does* learn from exploratory moves is better — it correctly values the policy we actually play, so it wins more. If exploration will stop, the greedy-only values are preferred.

Session 2 — Multi-Armed Bandits & Policy Gradient

Q1 · Module 2 review Q1

Explain why RL feedback is evaluative not instructive, with an example.

✓ Solution

Instructive feedback names the correct action regardless of what you did; evaluative feedback only scores the action you took. Pulling a slot machine for Rs. 5 doesn't reveal that another arm pays Rs. 10 — hence the need to explore.

Q2 · Module 2 review Q2

Why is the multi-armed bandit an RL problem in a non-associative setting?

✓ Solution

There is a single situation (no state to associate actions with). The same k actions recur each step and rewards don't depend on a state, so the agent only learns which action is best on average — full RL minus the sequential/state-credit part, under the explore-exploit trade-off.

Q3 · Numeric

ε -greedy, four actions (a greedy), $\varepsilon = 0.4$: probability greedy action is chosen, and each non-greedy action?

✓ Solution

$P(\text{greedy}) = (1 - \varepsilon) + \varepsilon/k = 0.6 + 0.4/4 = \mathbf{0.7}$; each non-greedy $= \varepsilon/k = 0.4/4 = \mathbf{0.1}$.
Check: $0.7 + 3(0.1) = 1.0$.

Q4 · Module 2 review Q8

Step size $1/n$ vs constant α : effect on (i) non-stationarity and (ii) high reward variance?

✓ Solution

$1/n$ (sample average) weights all rewards equally — averages out noise well but adapts slowly (poor for non-stationary). Constant α gives an exponential recency-weighted average — tracks a drifting target (good for non-stationary) but stays jittery under high variance.

Q5 · Module 2 review Q9

Why is the effect of optimistic initial values temporary, and when do you use it?

✓ Solution

High initial Q makes every tried action look disappointing, forcing early exploration of all actions. As each is sampled its estimate falls to the truth and the optimism wears off. Use it as a simple exploration kick-start for stationary problems; it is poor for non-stationary ones.

Q6 · Module 2 review Q14

How does UCB differ from ε -greedy?

✓ Solution

ε -greedy explores uniformly at random; UCB explores by uncertainty, choosing $\arg \max_a [Q(a) + c\sqrt{\ln t / N(a)}]$. The bonus is large for rarely-tried actions and shrinks as $N(a)$ grows, so UCB explores more intelligently.

Q7 · Exam EC-2 Q3(a), 3 marks, numeric

Observations (t, a, r) : $(1, S, 7), (2, T, 5), (3, R, 6), (4, C, 4), (5, S, 8), (6, T, 6), (7, R, 7), (8, C, 5)$. With recency-weighted update $\alpha = 0.5$, $Q_0 = 0$, find the best intervention.

✓ Solution

$Q \leftarrow Q + 0.5(R - Q)$ per action:

$Q(S) : 0 \rightarrow 3.5 \rightarrow 3.5 + 0.5(8 - 3.5) = \mathbf{5.75}$; $Q(T) : 0 \rightarrow 2.5 \rightarrow \mathbf{4.25}$;

$Q(R) : 0 \rightarrow 3.0 \rightarrow \mathbf{5.00}$; $Q(C) : 0 \rightarrow 2.0 \rightarrow \mathbf{3.50}$.

Best = S (SMS reminder), $Q(S) = 5.75$.

Q8 · Exam EC-3 Q1(a), 4 marks, numeric

Actions M, T, rewards ± 1 . M chosen $4 \times \{+1, -1, +1, r\}$, T $4 \times \{+1, +1, -1, -1\}$. With UCB ($c = 1$) at step 9, max r for which $UCB(T) > UCB(M)$?

✓ Solution

Both were pulled $N = 4$ times, so the bonus $c\sqrt{\ln t / N}$ is equal and cancels. $Q(T) = (1+1-1-1)/4 = 0$; $Q(M) = (1+r)/4$. $UCB(T) > UCB(M) \iff 0 > (1+r)/4 \iff r < -1$. At $r = -1$ they tie; with rewards bounded at ± 1 , Therapy can never strictly win.

Q9 · CS2-3 slides

Write the softmax action-selection probability and describe the gradient-bandit update.

✓ Solution

$\pi(a) = e^{H(a)} / \sum_b e^{H(b)}$. Gradient ascent on expected reward: $H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t))$, with non-chosen preferences moved oppositely. It learns relative preferences (not action-values) and explores via the softmax.

Q10 · Module 2 review Q3

Give one application modellable as a multi-armed bandit and explain why.

✓ Solution

Online ad / article selection (A/B testing): each ad is an arm, showing it yields a stochastic reward (click / no click). It fits MAB because the same arms recur each round, there is essentially no state (the choice doesn't change the world), and we must balance showing the current-best ad (exploit) vs gathering data on others (explore). Non-associative, single-state, evaluative feedback.

Q11 · Module 2 review Q4

If the action value is the same for all actions, do you still need to explore?

✓ Solution

If you *know* all values are equal, exploration gains nothing — any action is optimal. But if the equality is only your current *estimate*, you must still explore to reduce uncertainty and confirm none is actually better; the danger is mistaking ignorance for knowledge.

Q12 · Module 2 review Q5

Tabulate the exploration–exploitation approaches with a remark on each.

✓ Solution

ϵ -greedy — explore uniformly w.p. ϵ ; simple but blind. Decaying ϵ — explore less over time; good for stationary problems. Optimistic initial values — high initial Q forces early trials; one-off, stationary only. UCB — bonus $c\sqrt{\ln t / N(a)}$ for under-tried actions; explores by uncertainty. Gradient/softmax — sample by preference; smooth, temperature-controlled exploration.

Q13 · Module 2 review Q6

Choose ϵ for (a) deterministic rewards, (b) low-variance, (c) high-variance. Why?

✓ Solution

(a) $\epsilon \approx 0$: one sample reveals each true value. (b) Small ϵ : a few samples suffice. (c) Larger ϵ : noisy rewards need many samples to tell arms apart. In short, more reward noise $\Rightarrow$ more exploration.

Q14 · Module 2 review Q12

How do policy-based methods differ from value-based methods?

✓ Solution

Value-based (ϵ -greedy on Q , Q -learning): learn values, derive the policy by acting greedily. Policy-based (gradient bandit, policy gradient): learn the policy directly by gradient ascent on expected reward. Policy-based handles stochastic optimal policies and continuous actions naturally; value-based is simpler when a greedy policy suffices.

Q15 · Module 2 review Q13

Name two approaches to maintain action-value estimates.

✓ Solution

- (1) Sample average $Q_n = \frac{1}{n} \sum R_i$ (step $1/n$) — equal weights, for stationary problems.
- (2) Exponential recency-weighted average with constant step α : $Q \leftarrow Q + \alpha(R - Q)$ — recent rewards weigh more, for non-stationary problems. Both fit $\text{New} = \text{Old} + \text{StepSize} \cdot [\text{Target} - \text{Old}]$.

Q16 · Module 2 review Q15

What factors decide the right ϵ for ϵ -greedy?

✓ Solution

Reward variance/noise (more $\Rightarrow$ larger ϵ), stationarity (drift $\Rightarrow$ don't decay to 0), number of actions (more arms $\Rightarrow$ more exploration), time horizon (short $\Rightarrow$ explore less), and the cost of a mistake. Balance: explore enough to find the best arm, not so much that you forgo reward.

Q17 · Exam EC-2 Q1(b), 2 marks

Wristband: Case 1 — mode choice doesn't affect future rewards; Case 2 — long use of Mode C raises battery temperature, lowering future power-hungry rewards. Classify each.

✓ Solution

Case 1: actions don't influence future situations $\Rightarrow$ **Multi-Armed Bandit** (non-associative, no transitions). Case 2: the action changes the future (temperature) and hence future rewards $\Rightarrow$ **finite MDP**; the added dependency introduces state, so it is no longer a bandit.

Q18 · Exam EC-2 Q3(b), 1.5 marks

In $Q \leftarrow Q + \alpha(R - Q)$ with $\alpha = 0.5$, explain α . What if $\alpha = 1$?

✓ Solution

α sets how fast new rewards overwrite the estimate; $\alpha = 0.5$ gives an exponentially-decaying memory (recent rewards weigh more) — good for non-stationary settings. If $\alpha = 1$, $Q \leftarrow R$: the estimate equals only the last reward, ignoring history — maximally reactive but very noisy.

Q19 · Exam EC-2 Q3(c), 1 mark

What is the significance of the confidence term c in UCB?

✓ Solution

In $Q(a) + c\sqrt{\ln t / N(a)}$, c scales the exploration bonus — the optimism under uncertainty. Larger $c \Rightarrow$ more weight on poorly-sampled actions (more exploration); smaller $c \Rightarrow$ more greedy. It tunes how aggressively UCB probes uncertain options before committing.

Q20 · Exam EC-2 Q3(d), 2 marks, numeric

Observations $(1, S, 7), (2, T, 5), (3, R, 6), (4, C, 4), (5, S, 8)$ under sample-average greedy. On which steps did an ε -random action *definitely* occur, and on which *possibly*?

✓ Solution

Greedy = $\arg \max Q$ before each step (all $Q = 0$ initially): t1 all-tie, S could be greedy $\rightarrow$ possibly; t2 greedy=S(7) but chose T $\rightarrow$ definitely; t3 greedy=S but chose R $\rightarrow$ definitely; t4 greedy=S but chose C $\rightarrow$ definitely; t5 greedy=S and chose S $\rightarrow$ possibly. **Definitely random: t=2,3,4. Possibly random: t=1 and 5.**

Q21 · Exam EC-3 Q1(b), 4 marks, numeric

Non-stationary update $\alpha = 0.5$, $Q_1(M) = 0$. M rewards: $+1(t1), -1(t2), +1(t6), r(t8)$. Compute $Q(M)$ at $t=9$.

✓ Solution

Apply $Q \leftarrow Q + 0.5(R - Q)$: t1: 0.5; t2: $0.5 + 0.5(-1 - 0.5) = -0.25$; t6: $-0.25 + 0.5(1 + 0.25) = 0.375$; t8: $0.375 + 0.5(r - 0.375) = 0.1875 + 0.5r$. **$Q(M)$ at $t=9 = 0.5r + 0.1875$.**

Q22 · Exam EC-3 Q1(c), 2 marks

Comment on $Q(M)$ and its sensitivity to r .

✓ Solution

With $\alpha = 0.5$, the latest reward r accounts for half of $Q(M)$ (coefficient 0.5), so the estimate — and whether Medication beats Therapy — is highly sensitive to the single most recent outcome. Large α reacts sharply to the newest reward: good for tracking change, risky if that reward is noisy.

Session 3 — Markov Decision Processes & Dynamic Programming

Q1 · Core

List the components of a finite MDP.

✓ Solution

States S , actions A , transition model $p(s' | s, a)$, reward $r(s, a, s')$, discount $\gamma \in [0, 1]$. Markov property: next state/reward depend only on the current state and action.

Q2 · Exam EC-3 Q2(a), 2 marks, numeric

Robot at M, $\pi(\text{Continue} \mid M) = 1$: $\rightarrow M$ ($p=0.6, r=+1$), $\rightarrow L$ ($p=0.4, r=+1$), $\gamma = 0.9$, $V(M) = V(L) = 1$. Bellman expectation $V_\pi(M)$?

✓ Solution

$$V_\pi(M) = \sum_{s'} p(s' \mid M, C)[r + \gamma V(s')] = 0.6(1 + 0.9 \cdot 1) + 0.4(1 + 0.9 \cdot 1) = 0.6(1.9) + 0.4(1.9) = 1.9.$$

Q3 · Exam EC-3 Q2(b), 4 marks, numeric

With $V^0(L) = -10, V^0(M) = 5, V^0(H) = 8, \gamma = 0.9$: compute $Q(M, \text{Continue})$ and $Q(M, \text{Recharge})$ ($\text{Recharge} \rightarrow H, p=1, r=-2$). Does the policy at M change?

✓ Solution

$$Q(M, C) = 0.6(1 + 0.9 \cdot 5) + 0.4(1 + 0.9(-10)) = 0.6(5.5) + 0.4(-8) = 3.3 - 3.2 = \mathbf{0.10}.$$

$$Q(M, R) = 1.0(-2 + 0.9 \cdot 8) = -2 + 7.2 = \mathbf{5.20}.$$

$5.20 > 0.10$, so the policy switches: $\pi_1(\text{Recharge} \mid M) = 1$.

Q4 · Core

Contrast value iteration and policy iteration.

✓ Solution

Value iteration repeatedly applies the Bellman optimality backup $V(s) \leftarrow \max_a \sum p(s' \mid s, a)[r + \gamma V(s')]$ until V converges, then extracts the greedy policy. Policy iteration alternates full policy evaluation (solve V_π) and policy improvement (greedy w.r.t. V_π) until the policy stops changing. VI = many cheap sweeps; PI = fewer heavier iterations.

Q5 · Exam EC-2 Q2(d)

What does γ do, and what is the impact of $\gamma = 0.01$ on a task that must keep a patient stable?

✓ Solution

γ weights future reward in $G_t = \sum_k \gamma^k R_{t+k+1}$. A tiny γ makes the agent myopic: it discounts anything beyond a step or two, chasing short-term gains and failing to value the long sequence needed to keep the patient stable — dangerous for a long-horizon goal.

Q6 · Exam EC-2 Q4(a), 2 marks

In model-free control, why is estimating only $v_\pi(s)$ insufficient, even if perfect?

✓ Solution

Greedy action selection needs $\arg \max_a \sum_{s'} p(s' \mid s, a)[r + \gamma v_\pi(s')]$, which requires the model p . Model-free you don't have it, so $v_\pi(s)$ alone can't rank actions. Learn $q_\pi(s, a)$ instead — its $\arg \max_a$ gives the best action without a model.

Q7 · Exam-style (robot), numeric

One synchronous VI sweep on H (Continue: $\rightarrow H \ p=0.7 \ r=+2$, $\rightarrow M \ p=0.3 \ r=+2$; Recharge: $\rightarrow H \ p=1 \ r=-1$), $V(H)=V(M)=1$, $\gamma = 0.9$. New $V(H)$?

✓ Solution

$Q(H, C) = 0.7(2+0.9)+0.3(2+0.9) = 0.7(2.9)+0.3(2.9) = 2.9$; $Q(H, R) = -1+0.9 = -0.1$.
 $V(H) \leftarrow \max(2.9, -0.1) = \mathbf{2.9}$; greedy action at H is Continue.

Q8 · Exam EC-2 Q2(a), 2.5 marks

An ICU ventilator assistant categorises SpO_2 as lowO2 / OptimalO2 / highO2 and may increase / decrease / maintain pressure; increase/decrease leaves health unchanged w.p. 0.4; maintain always keeps the condition; four consecutive Optimal readings $\Rightarrow$ stabilised. Formulate the MDP.

✓ Solution

States: {lowO2, OptimalO2, highO2} plus a *stabilised* terminal state (really (category, #consecutive-Optimal)). **Actions:** {increase, decrease, maintain}. **Transitions:** increase/decrease change the level w.p. 0.6, unchanged w.p. 0.4 (split remaining mass equally where unspecified); maintain keeps the state w.p. 1. **Reward:** + for staying/moving to Optimal, - for over-/under-ventilation. $\gamma < 1$. Goal: reach the stabilised state while avoiding over-ventilation.

Q9 · Exam EC-2 Q2(b), 2 marks

Design A: +10 for lowO2 $\rightarrow$ OptimalO2. Design B: -10 for OptimalO2 $\rightarrow$ highO2. Which do you prefer, in terms of (i) behaviour encouraged and (ii) when each is preferable?

✓ Solution

(i) A positively reinforces recovering a hypoxic patient to Optimal; B punishes over-ventilation past Optimal. (ii) Prefer B when the dominant risk is over-ventilation harm (penalising the dangerous transition is safer); prefer A when the priority is actively driving hypoxic patients back to Optimal. A combined shaped reward (reward Optimal, penalise highO2) is best in practice.

Q10 · Exam EC-2 Q2(c),(d), 3 marks

Is the ventilator task episodic or continuing? Impact of $\gamma = 0.01$?

✓ Solution

Episodic — a terminal stabilised state (4 consecutive Optimal readings) ends each patient's episode. With $\gamma = 0.01$ the agent is extremely myopic: it values only the immediate reading and ignores the multi-step path to stabilisation, so it may oscillate or fail to plan the needed adjustments — endangering stability.

Session 4 — Monte Carlo Methods

Q1 · Core

What is first-visit Monte-Carlo policy evaluation, and how does it differ from dynamic programming?

✓ Solution

Run complete episodes under π ; for each state average the returns G_t following its *first* visit per episode. It is model-free (needs no p) and does not bootstrap (uses the actual full return). DP plans with a known model; MC learns from sampled experience.

Q2 · Exam EC-3 Q2(c), numeric

Episode $(M,C,+1,M) \rightarrow (M,C,+1,L) \rightarrow (L,R,-3,M) \rightarrow (M,C,+1,\text{term})$, $\gamma = 0.9$. First-visit $V(M)$?

✓ Solution

First visit to M is at $t = 0$: $G = 1 + 0.9(1) + 0.9^2(-3) + 0.9^3(1) = 1 + 0.9 - 2.43 + 0.729 = \mathbf{0.199}$. So $V(M) \approx 0.199$ (later visits to M are ignored in first-visit MC).

Q3 · Exam EC-2 Q4(b), 2 marks

Why may first-visit MC with a deterministic policy fail to reach the optimal policy? One precise fix.

✓ Solution

A deterministic policy always takes the same action per state, so many (s, a) pairs are never visited and never evaluated — the greedy step can't discover better actions. Fix: **exploring starts** (begin episodes at random (s, a) pairs) or use an ϵ -soft policy.

Q4 · Core

Distinguish on-policy and off-policy learning.

✓ Solution

On-policy: learn about the policy you act with (behaviour = target). Off-policy: act with an exploratory behaviour policy b but learn about a different target π (often greedy). Off-policy is flexible but needs importance sampling to correct the b vs π mismatch.

Q5 · Exam EC-3 Q2(d), numeric

$b(C|M)=0.7, b(R|M)=0.3$; target $\pi(C|M)=1$. Episode $(M,C,+1,M) \rightarrow (M,R,-2,H) \rightarrow (H,C,+2,\text{term})$, $\gamma = 0.9$. Importance-sampling ratio ρ from first M, and return G ?

✓ Solution

$\rho = \prod_t \pi(A_t|S_t)/b(A_t|S_t)$. Step (M,C): $1/0.7$; step (M,R): $\pi(R|M)/b(R|M) = 0/0.3 = 0 \Rightarrow \rho = \mathbf{0}$ (target never recharges). $G = 1 + 0.9(-2) + 0.9^2(2) = 1 - 1.8 + 1.62 = \mathbf{0.82}$. In weighted IS this trajectory contributes nothing to $V(M)$.

Q6 · Core

Compare ordinary and weighted importance sampling.

✓ Solution

Both reweight returns by ρ . Ordinary IS divides by the count n : unbiased but high variance. Weighted IS divides by $\sum_i \rho_i$: biased (bias $\rightarrow 0$ as n grows) but far lower variance — preferred in practice.

Q7 · Exam EC-2 Q4(c), numeric

Chatbot episode $(s_0, a_1, 2) \rightarrow (s_0, a_3, 0) \rightarrow (s_1, a_2, 3) \rightarrow (s_1, a_2, -1, \text{term})$, $\gamma = 0.8$. First-visit MC return G for $V(s_0)$?

✓ Solution

First visit to s_0 at $t = 0$: $G = 2 + 0.8(0) + 0.8^2(3) + 0.8^3(-1) = 2 + 0 + 1.92 - 0.512 = \mathbf{3.408}$.

Worked Numericals (verified by direct computation)

N1 · Discounted return

A reward sequence after time t is $[1, 2, 3]$ with discount $\gamma = 0.5$. Compute the return G_t , and compare with the reversed sequence $[3, 2, 1]$.

✓ Solution

$G_t = \sum_k \gamma^k R_{t+k+1}$. For $[1, 2, 3]$: $1 \cdot 1 + 0.5 \cdot 2 + 0.25 \cdot 3 = 1 + 1 + 0.75 = \mathbf{2.75}$. For $[3, 2, 1]$: $3 + 0.5 \cdot 2 + 0.25 \cdot 1 = 3 + 1 + 0.25 = \mathbf{4.25}$. So $G([1, 2, 3]) < G([3, 2, 1])$ — with discounting, front-loading the large rewards yields a higher return, which is why an agent prefers earning sooner.

N2 · Infinite-horizon return

An agent receives a constant reward $R = 1$ every step forever, with $\gamma = 0.9$. What is the return? What if $R = 2$ and $\gamma = 0.95$?

✓ Solution

A constant reward gives a geometric series $G = R(1 + \gamma + \gamma^2 + \dots) = \frac{R}{1-\gamma}$. For $R = 1, \gamma = 0.9$: $\frac{1}{0.1} = \mathbf{10}$. For $R = 2, \gamma = 0.95$: $\frac{2}{0.05} = \mathbf{40}$. Discounting keeps the infinite sum finite; the closer γ is to 1, the larger (and more far-sighted) the return.

N3 · Incremental action value

Action 2 of a bandit yields rewards 1, 2, 2 on its three pulls. Using $Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n)$ with Q_1 formed from the first reward, track the estimate.

✓ Solution

Start Q after pull 1 = 1. Pull 2 ($R = 2$): $Q = 1 + \frac{1}{2}(2 - 1) = 1.5$. Pull 3 ($R = 2$): $Q = 1.5 + \frac{1}{3}(2 - 1.5) = \mathbf{1.667}$. This equals the sample average $(1 + 2 + 2)/3 = 1.667$ — the incremental rule computes the running mean with $O(1)$ memory, never storing past rewards.

N4 · Epsilon-greedy

A 4-armed bandit uses ϵ -greedy with $\epsilon = 0.1$. What is the probability of selecting the current greedy action, and of each non-greedy action?

✓ Solution

With probability $1 - \epsilon$ the agent exploits the greedy action, and with ϵ it picks uniformly among all 4 arms. So $P(\text{greedy}) = (1 - \epsilon) + \epsilon/k = 0.9 + 0.1/4 = \mathbf{0.925}$, and each of the 3 other arms has $P = \epsilon/k = 0.1/4 = \mathbf{0.025}$. Check: $0.925 + 3(0.025) = 1.0$.

N5 · UCB selection

At step $t = 50$ (with $c = 1$), arm A has $Q = 0.5$, $N = 10$ pulls; arm B has $Q = 0.6$, $N = 2$ pulls. Which does UCB pick?

✓ Solution

UCB score = $Q + c\sqrt{\ln t / N}$. Arm A: $0.5 + \sqrt{\ln 50 / 10} = 0.5 + \sqrt{0.391} = 0.5 + 0.625 = \mathbf{1.125}$. Arm B: $0.6 + \sqrt{\ln 50 / 2} = 0.6 + \sqrt{1.956} = 0.6 + 1.399 = \mathbf{1.999}$. UCB picks **B**: although both have similar value estimates, B has been tried far less, so its large uncertainty bonus drives exploration toward it.

N6 · Bellman expectation

From state s a uniform-random policy picks action 1 (reward 1, deterministically to a state with $V = 5$) or action 2 (reward 0, to a state with $V = 10$). With $\gamma = 0.9$, compute $V_\pi(s)$.

✓ Solution

$V_\pi(s) = \sum_a \pi(a|s)[r + \gamma V(s')]$. Action 1 contributes $0.5(1 + 0.9 \cdot 5) = 0.5 \cdot 5.5 = 2.75$; action 2 contributes $0.5(0 + 0.9 \cdot 10) = 0.5 \cdot 9 = 4.5$. So $V_\pi(s) = 2.75 + 4.5 = \mathbf{7.25}$. The value is the policy-weighted average of immediate reward plus discounted successor value.

N7 · Value iteration

A 2-state chain: from A the only action moves to B with reward 0; from B the only action exits to a terminal state with reward 1. With $\gamma = 0.9$, find $V^*(A)$ and $V^*(B)$.

✓ Solution

Work backwards. $V^*(B) = 1 + \gamma \cdot V^*(\text{terminal}) = 1 + 0.9 \cdot 0 = \mathbf{1}$. $V^*(A) = 0 + \gamma \cdot V^*(B) = 0 + 0.9 \cdot 1 = \mathbf{0.9}$. Value iteration would reach the same fixed point: each sweep applies $V(s) = \max_a \sum_{s'} p(s'|s, a)[r + \gamma V(s')]$ until the values stop changing.

N8 · Softmax preferences

A gradient bandit has action preferences $H = [1.0, 2.0, 0.5]$. Compute the softmax selection probabilities.

✓ Solution

$\pi(a) = e^{H(a)} / \sum_b e^{H(b)}$. Exponentials: $e^{1.0} = 2.718$, $e^{2.0} = 7.389$, $e^{0.5} = 1.649$; sum = 11.756. Probabilities: $[2.718, 7.389, 1.649] / 11.756 = [\mathbf{0.231}, \mathbf{0.629}, \mathbf{0.140}]$ (sum 1). Action 2, with the highest preference, gets the largest share, but every action keeps a non-zero chance — built-in exploration.

N9 · TD(0) update

A state has estimate $V(s) = 6$. The agent takes a step, receives reward $r = 2$, and lands in s' with $V(s') = 10$. With $\gamma = 0.9$ and step size $\alpha = 0.1$, perform the TD(0) update.

✓ Solution

TD target = $r + \gamma V(s') = 2 + 0.9 \cdot 10 = 11$. TD error $\delta = \text{target} - V(s) = 11 - 6 = 5$. Update $V(s) \leftarrow V(s) + \alpha \delta = 6 + 0.1 \cdot 5 = \mathbf{6.5}$. TD bootstraps: it nudges $V(s)$ toward the one-step estimate after a single step, without waiting for the episode to end.

N10 · Monte Carlo return

An episode from state s yields the reward sequence $1, 0, 2$ until termination. With $\gamma = 1$, what return does first-visit Monte Carlo record for s ? How would a second episode giving return 5 change the estimate?

✓ Solution

With $\gamma = 1$ the return is the undiscounted sum from s : $G = 1 + 0 + 2 = \mathbf{3}$. First-visit MC stores this return for s . After a second episode returns 5, the MC estimate is the average of the recorded returns: $V(s) = (3 + 5)/2 = \mathbf{4}$. MC needs the complete episode because G is only known at termination.

Comprehensive Question Bank (50 Questions)

QB: Section A — RL Foundations

Q1 · Bank

Define reinforcement learning and how it differs from supervised and unsupervised learning.

✓ Solution

Reinforcement learning is learning what to do — mapping situations to actions — so as to maximise a numerical reward signal, by trial-and-error interaction with an environment. Unlike supervised learning there are no labelled correct actions; the agent only receives evaluative reward, often delayed. Unlike unsupervised learning the goal is not to find hidden structure but to maximise return. Its two distinctive features are trial-and-error search and delayed reward.

Q2 · Bank

Describe the agent-environment interaction loop and its core signals.

✓ Solution

At each time step t the agent observes a state S_t , chooses an action A_t , and the environment responds with a reward R_{t+1} and a next state S_{t+1} . This loop repeats, producing a trajectory. The agent's behaviour is its policy; the environment is everything outside the agent's control. Learning means improving the policy from the stream of (state, action, reward) experience.

Q3 · Bank

State the reward hypothesis.

✓ Solution

The reward hypothesis holds that all of what we mean by goals and purposes can be well thought of as the maximisation of the expected value of the cumulative sum of a received scalar signal (the reward). In other words, any goal can be encoded as a reward function, and acting to maximise expected cumulative reward is sufficient to pursue that goal.

Q4 · Bank

Explain the exploration-exploitation trade-off.

✓ Solution

To maximise reward the agent should exploit actions it already knows to be good; but to discover whether other actions are better it must explore actions it is uncertain about. Pure exploitation can lock onto a sub-optimal action; pure exploration wastes reward. The agent must balance the two — gathering enough information to act well without sacrificing too much reward along the way.

Q5 · Bank

Name the four main elements of an RL system.

✓ Solution

A policy (the agent's behaviour, mapping states to actions), a reward signal (the immediate goal, defining good and bad events), a value function (the long-run desirability of states, i.e. expected cumulative reward), and optionally a model of the environment (predicting next states and rewards). Reward is immediate; value is the prediction of total future reward.

Q6 · Bank

Distinguish reward from value, and explain why value is harder to obtain.

✓ Solution

Reward is the immediate signal received after an action; value is the total reward expected to accumulate from a state onward under a policy. We act to maximise value, not immediate reward — a low-reward move may lead to high future reward. Value is harder because it must be estimated from sequences of observations over time, whereas reward is given directly by the environment.

Q7 · Bank

Differentiate episodic from continuing tasks.

✓ Solution

Episodic tasks break into finite episodes ending in a terminal state (a game, a maze run), after which the environment resets; returns are summed to the episode end. Continuing tasks go on indefinitely with no natural terminal state (process control), so returns must be discounted to stay finite. Discounting unifies the two: an episodic task is a continuing one that transitions to an absorbing zero-reward state.

Q8 · Bank

Contrast model-based and model-free reinforcement learning.

✓ Solution

Model-based RL learns or is given a model of the environment (transition and reward functions) and plans with it (e.g. dynamic programming), which is sample-efficient but needs an accurate model. Model-free RL learns values or policies directly from experience without a model (e.g. Monte Carlo, temporal-difference), which is simpler and robust to model error but needs more interaction. Many systems combine both.

Q9 · Bank

What is the difference between on-policy and off-policy learning?

✓ Solution

On-policy methods evaluate and improve the same policy that is used to generate behaviour (the behaviour and target policies are identical). Off-policy methods learn about a target policy different from the behaviour policy that generates the data — for example learning the greedy optimal policy while behaving with an exploratory policy. Off-policy is more flexible but often higher-variance and needs corrections like importance sampling.

Q10 · Bank

Give three application domains of RL and what makes them suitable.

✓ Solution

Game playing (Go, Atari, chess) — clear reward, simulatable, sequential decisions; robotics and control — continuous interaction with delayed consequences; recommendation, operations and resource management (inventory, energy, traffic) — sequential choices that affect future state. RL fits problems framed as sequential decision-making under uncertainty with a definable reward and the ability to interact or simulate.

QB: Section B — Multi-Armed Bandits & Action Values**Q11 · Bank**

Define the k-armed bandit problem.

✓ Solution

You repeatedly choose among k actions (arms); each yields a numerical reward drawn from a stationary probability distribution that depends only on the action chosen. There is a single state (no transitions), so the goal is simply to maximise total expected reward over many steps by learning which arm has the highest mean reward. It isolates the exploration-exploitation problem from the credit-assignment problem of full RL.

Q12 · Bank

What is an action value $Q(a)$, and how is it estimated by sample averaging?

✓ Solution

The action value $Q^*(a)$ is the true expected reward of taking action a. The sample-average estimate is the mean of the rewards actually received from a: $Q_n(a) = (\text{sum of rewards from a}) / (\text{number of times a was taken})$. By the law of large numbers it converges to $Q^*(a)$ as the count grows. It is the simplest unbiased estimate of an action's worth.

Q13 · Bank

Give the incremental update rule for action values and why it is used.

✓ Solution

$Q_{n+1} = Q_n + (1/n)(R_n - Q_n)$: the new estimate is the old plus a step size times the error (reward minus old estimate). It computes the running sample average without

storing all past rewards (constant memory and computation per step). The general form $Q \leftarrow Q + \alpha(R - Q)$ uses a step size α , the template for almost all RL update rules.

Q14 · Bank

Contrast a sample-average step size with a constant step size, and when each is used.

✓ Solution

A sample-average step ($\alpha = 1/n$) weights all rewards equally and converges to the true mean — ideal for a stationary problem. A constant step (α fixed in $(0,1]$) makes the estimate an exponentially weighted recency-weighted average, giving more weight to recent rewards — essential for non-stationary problems where the true values drift, because the estimate keeps tracking the change instead of freezing.

Q15 · Bank

Explain epsilon-greedy action selection.

✓ Solution

With probability $1 - \epsilon$ the agent exploits — picks the action with the highest current estimate; with probability ϵ it explores — picks an action uniformly at random. So with k actions the greedy action is chosen with probability $1 - \epsilon + \epsilon/k$ and each other with ϵ/k . A larger ϵ explores more; ϵ is often decayed over time to exploit more as estimates improve.

Q16 · Bank

What is optimistic initial values and how does it encourage exploration?

✓ Solution

Initialise the action-value estimates far higher than any realistic reward (optimistically). Then whichever action is tried returns a reward lower than the estimate, so its value drops and the agent is 'disappointed' into trying other still-optimistic actions. This drives systematic early exploration even with a greedy policy — though it is a temporary effect suited to stationary problems.

Q17 · Bank

Describe Upper-Confidence-Bound (UCB) action selection.

✓ Solution

UCB picks the action maximising $Q_t(a) + c \cdot \sqrt{\ln t / N_t(a)}$: the estimated value plus an exploration bonus that grows with total time t and shrinks with the number of times a has been tried $N_t(a)$. It explores actions whose estimates are uncertain (rarely tried) rather than uniformly at random, focusing exploration where it is most informative. The constant c controls the degree of exploration.

Q18 · Bank

Explain the gradient-bandit algorithm and the softmax it uses.

✓ Solution

Instead of estimating values, the gradient bandit learns a numerical preference $H(a)$ for each action and selects actions by a softmax (Boltzmann) distribution: $\pi(a) = \exp(H(a)) / \sum_b \exp(H(b))$. Preferences are updated by stochastic gradient ascent on expected reward, increasing the preference of actions that beat a baseline (the average reward) and decreasing the others. Only relative preferences matter, not absolute values.

Q19 · Bank

What is regret in a bandit problem?

✓ Solution

Regret is the expected loss from not always playing the optimal arm: the difference between the reward an oracle that always plays the best arm would accumulate and the reward the algorithm actually accumulates. Good bandit algorithms achieve regret that grows only logarithmically with the number of steps, meaning the per-step loss vanishes over time as the best arm is identified.

Q20 · Bank

Why is the bandit setting a useful simplification of full RL?

✓ Solution

A bandit has a single state and no transitions, so actions do not change the situation — there is no long-term credit assignment, only immediate reward. This isolates the exploration-exploitation problem cleanly, letting us study action-value estimation and exploration strategies (epsilon-greedy, UCB, gradient) before adding the complexity of states, transitions and delayed reward in full Markov decision processes.

QB: Section C — MDPs, Returns & Value Functions

Q21 · Bank

Define a Markov Decision Process and its components.

✓ Solution

An MDP is the formal model of sequential decision-making: a tuple (S, A, P, R, γ) of states S , actions A , a transition function $P(s'|s,a)$, a reward function $R(s,a,s')$, and a discount factor γ in $[0,1]$. It generalises a bandit by adding states and transitions, so actions affect not only immediate reward but also which states are visited next.

Q22 · Bank

State the Markov property and why it matters.

✓ Solution

The Markov property says the future depends only on the current state, not on the full history: $P(S_{t+1}, R_{t+1} | S_t, A_t)$ is unchanged by earlier states and actions. It matters because it lets the state summarise everything relevant for decision-making, so a policy can be a function of the current state alone and value functions obey recursive (Bellman) equations.

Q23 · Bank

Define the return G_t , and give the discounted form.

✓ Solution

The return is the cumulative future reward the agent seeks to maximise. The discounted return is $G_t = R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \dots = \sum_{k \geq 0} \gamma^k \cdot R_{t+k+1}$. The discount factor γ weights nearer rewards more heavily; the agent's objective is to maximise the expected return from each state.

Q24 · Bank

What is the role of the discount factor γ ?

✓ Solution

γ in $[0,1)$ trades off immediate versus future reward and keeps the return finite for continuing tasks (a bounded reward sum converges because γ^k decays). Near 0 makes the agent myopic (cares about immediate reward); near 1 makes it far-sighted. It also reflects uncertainty about the future and helps algorithms converge. With $\gamma = 1$ only episodic, terminating tasks have finite returns.

Q25 · Bank

Define the state-value function V_π and the action-value function Q_π .

✓ Solution

$V_\pi(s)$ is the expected return starting from state s and following policy π thereafter: $V_\pi(s) = E_\pi[G_t | S_t = s]$. $Q_\pi(s,a)$ is the expected return starting from s , taking action a , then following π : $Q_\pi(s,a) = E_\pi[G_t | S_t = s, A_t = a]$. Q is finer-grained: it values each action in a state, which is what we need to improve the policy.

Q26 · Bank

Write the Bellman expectation equation for V_π and explain it.

✓ Solution

$V_\pi(s) = \sum_a \pi(a|s) \cdot \sum_{s',r} p(s',r|s,a) \cdot [r + \gamma \cdot V_\pi(s')]$. It says the value of a state equals the immediate reward plus the discounted value of the successor state, averaged over the policy's actions and the environment's transitions. It is a self-consistency (recursive) condition relating each state's value to its successors'.

Q27 · Bank

What makes a policy optimal, and what is the optimal value function?

✓ Solution

A policy π^* is optimal if its expected return is at least that of every other policy in every state. All optimal policies share the same optimal state-value function $V^*(s) = \max_{\pi} V_{\pi}(s)$ and optimal action-value function $Q^*(s,a)$. There is always at least one deterministic optimal policy, obtainable greedily from Q^* .

Q28 · Bank

State the Bellman optimality equation and how it differs from the expectation equation.

✓ Solution

$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a) \cdot [r + \gamma \cdot V^*(s')]$. It replaces the policy-average (sum over a of $\pi(a|s)$) with a maximum over actions, expressing that the optimal value of a state is the value of its best action. The expectation equation evaluates a fixed policy; the optimality equation characterises the best achievable value and is non-linear because of the max.

Q29 · Bank

How do you derive an optimal policy from V^* or Q^* ?

✓ Solution

From Q^* it is immediate: $\pi^*(s) = \arg\max_a Q^*(s,a)$ — act greedily with respect to the optimal action values. From V^* you need a one-step look-ahead using the model: $\pi^*(s) = \arg\max_a \sum_{s',r} p(s',r|s,a) \cdot [r + \gamma \cdot V^*(s')]$. Greedy action selection with respect to the optimal values yields an optimal policy.

Q30 · Bank

Why are episodic and continuing returns unified by an absorbing state?

✓ Solution

An episodic task can be made to look continuing by adding a special absorbing terminal state that transitions only to itself with reward 0. After termination, all further rewards are 0, so the infinite discounted sum equals the finite episode return. This lets a single discounted-return formula (and the same algorithms) handle both episodic and continuing tasks.

QB: Section D — Dynamic Programming

Q31 · Bank

What is dynamic programming in the RL context, and what does it require?

✓ Solution

Dynamic programming is a family of algorithms that compute optimal policies given a perfect model of the environment as an MDP (known transition and reward functions). It uses the Bellman equations as update rules, turning them into iterative computations. Its requirements — a complete, accurate model and enough computation to sweep the state space — are also its main limitations.

Q32 · Bank

Explain iterative policy evaluation.

✓ Solution

Policy evaluation computes V_{π} for a given policy by turning the Bellman expectation equation into an update applied repeatedly: $V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V_k(s')]$, sweeping all states each iteration. The sequence V_k converges to V_{π} as k grows (a contraction mapping with $\gamma < 1$). It answers 'how good is this policy?'.

Q33 · Bank

What is policy improvement and the policy improvement theorem?

✓ Solution

Given V_{π} , policy improvement makes a new policy greedy with respect to it: $\pi'(s) = \operatorname{argmax}_a Q_{\pi}(s,a)$. The policy improvement theorem guarantees that π' is at least as good as π ($V_{\pi'} \geq V_{\pi}$ everywhere), and strictly better unless π was already optimal. Acting greedily on a policy's own value can only help.

Q34 · Bank

Describe policy iteration.

✓ Solution

Policy iteration alternates two steps until convergence: evaluate the current policy to get V_{π} (policy evaluation), then improve the policy greedily with respect to V_{π} (policy improvement). Each round yields a strictly better policy, and since a finite MDP has finitely many policies, it converges to an optimal policy in a finite number of iterations.

Q35 · Bank

Describe value iteration and how it relates to policy iteration.

✓ Solution

Value iteration turns the Bellman optimality equation into an update: $V_{k+1}(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V_k(s')]$, sweeping until V converges to V^* , then extracts the greedy policy. It is policy iteration with policy evaluation truncated to a single sweep — combining evaluation and improvement into one step, often faster in practice.

Q36 · Bank

What is generalised policy iteration (GPI)?

✓ Solution

GPI is the general idea that evaluation and improvement processes interact, each pushing toward consistency: evaluation makes the value function consistent with the current policy, improvement makes the policy greedy with respect to the value function. Almost all RL methods are forms of GPI; they converge when the value function and policy stop changing — at the optimal pair.

Q37 · Bank

What is bootstrapping in dynamic programming?

✓ Solution

Bootstrapping means updating an estimate using other estimates rather than only actual returns. DP updates $V(s)$ from the current estimates of successor values $V(s')$, not from complete sampled returns. This makes updates cheap and allows learning before an episode ends, but introduces bias until the estimates converge. Temporal-difference learning also bootstraps; Monte Carlo does not.

Q38 · Bank

Why does dynamic programming converge?

✓ Solution

The Bellman optimality (and expectation) operator is a contraction mapping in the max-norm with factor $\gamma < 1$: each application brings any two value estimates strictly closer together. By the Banach fixed-point theorem it has a unique fixed point (V^* or V_π) to which repeated application converges geometrically. This guarantees both policy evaluation and value iteration converge.

Q39 · Bank

What is asynchronous dynamic programming and why is it useful?

✓ Solution

Asynchronous DP updates states in any order, one or a few at a time, rather than sweeping the entire state set each iteration. It still converges as long as every state continues to be updated, and it can focus computation on the states that matter most (e.g. those on likely trajectories), avoiding wasted effort on irrelevant states — important for large state spaces.

Q40 · Bank

What are the limitations of dynamic programming?

✓ Solution

DP needs a complete, accurate model of the environment, which is often unavailable. It also suffers the curse of dimensionality: the number of states grows exponentially with the number of state variables, so full sweeps become infeasible for large problems. These limits motivate model-free, sample-based methods (Monte Carlo and temporal-difference) that learn from experience.

QB: Section E — Monte Carlo Methods**Q41 · Bank**

What is Monte Carlo prediction, and how does it differ from DP?

✓ Solution

Monte Carlo prediction estimates a value function by averaging the actual returns observed after visiting each state, using complete episodes of experience. Unlike DP it needs no model of the environment — it learns directly from sampled episodes — and it does not bootstrap: each estimate uses a full return rather than other estimates. It only applies to episodic tasks (episodes must terminate).

Q42 · Bank

Distinguish first-visit from every-visit Monte Carlo.

✓ Solution

First-visit MC averages the returns following only the first time a state is visited in each episode; every-visit MC averages the returns following every visit to the state within each episode. Both converge to V_{π} as the number of visits grows; first-visit gives independent samples with cleaner theory, while every-visit reuses more data per episode.

Q43 · Bank

Why must Monte Carlo wait until the end of an episode?

✓ Solution

Because it updates values using the actual return G_t , which is the sum of rewards from time t to the terminal state — that quantity is only known once the episode has finished. So MC cannot update on the fly during an episode (unlike TD learning, which bootstraps from the next estimate after a single step) and is unsuitable for continuing tasks that never terminate.

Q44 · Bank

What is the exploring-starts assumption in Monte Carlo control?

✓ Solution

Exploring starts requires every (state, action) pair to have a non-zero probability of being the start of an episode, so that all actions in all states are eventually sampled even under a greedy policy. It guarantees the action values for every action are estimated, enabling

correct policy improvement — but it is often impractical, which motivates epsilon-soft on-policy and off-policy alternatives.

Q45 · Bank

Why does Monte Carlo control estimate action values Q rather than state values V ?

✓ Solution

Without a model you cannot do the one-step look-ahead that turns V into a policy (it needs transition probabilities). Action values $Q(s,a)$ let you improve the policy directly by acting greedily — $\pi(s) = \operatorname{argmax}_a Q(s,a)$ — with no model. So model-free control estimates Q , the value of each action in each state, rather than V .

Q46 · Bank

Describe on-policy epsilon-soft Monte Carlo control.

✓ Solution

It evaluates and improves an epsilon-soft policy (every action has at least $\epsilon/|A|$ probability), so exploration is built into the policy itself and exploring starts are unnecessary. Policy improvement makes the policy epsilon-greedy with respect to the current Q . It converges to the best epsilon-soft policy — slightly sub-optimal because it never stops exploring, but practical.

Q47 · Bank

What is off-policy learning via importance sampling in Monte Carlo?

✓ Solution

Off-policy MC learns the value of a target policy π while following a different behaviour policy b . Because the returns were generated under b , each return is reweighted by the importance-sampling ratio — the product over the episode of $\pi(a|s)/b(a|s)$ — to correct the distribution mismatch, so the weighted returns estimate values under π . The behaviour policy must cover the target ($b > 0$ wherever $\pi > 0$).

Q48 · Bank

Contrast ordinary and weighted importance sampling.

✓ Solution

Ordinary importance sampling averages the ratio-scaled returns by dividing by the number of episodes; it is unbiased but can have very high (even infinite) variance. Weighted importance sampling divides by the sum of the ratios instead; it is biased (the bias vanishes asymptotically) but has much lower variance and is strongly preferred in practice. It trades a little bias for a large variance reduction.

Q49 · Bank

Compare Monte Carlo with temporal-difference (TD) learning.

✓ Solution

MC waits for the full return G_t (no bootstrapping), so it is unbiased but high-variance and only works for episodic tasks. TD updates after one step using the estimate of the next state ($V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$), so it bootstraps, can learn online before an episode ends and on continuing tasks, and is lower-variance but biased. TD usually learns faster in practice; both are model-free.

Q50 · Bank

What are the advantages and limitations of Monte Carlo methods?

✓ Solution

Advantages: no model needed (learns from experience or simulation), unbiased estimates, can focus on a subset of states, and is unaffected by violations of the Markov property. Limitations: requires episodic tasks that terminate, must wait until episode end to update (no online learning), can be high-variance, and needs sufficient exploration (exploring starts or epsilon-soft/off-policy) to estimate all action values.

Official Review Questions — Worked Solutions

Review: Module 1 — Introduction to RL

Q1 · Review

Define reinforcement learning in your own words, and critically inspect the definition.

✓ Solution

RL is learning *what to do* — mapping situations to actions — so as to maximise a numerical reward signal, by trial-and-error interaction with an environment, where feedback is evaluative and often delayed. A good definition must mention: (i) sequential interaction (state, action, reward), (ii) a goal expressed as cumulative reward, and (iii) learning from experience rather than from labelled examples. A definition that omits the *cumulative/long-term* aspect is incomplete, since RL optimises return, not immediate reward.

Q2 · Review

What are the elements of RL and the purpose of each?

✓ Solution

Policy — the agent's behaviour, mapping states to actions. **Reward signal** — defines the immediate goal (good/bad events). **Value function** — long-run desirability of a state (expected cumulative reward), used to choose actions. **Model (optional)** — predicts next state and reward, enabling planning. Reward is immediate; value is the prediction we actually act on.

Q3 · Review

Which RL elements are vital to defining the problem versus the solution?

✓ Solution

The *problem* is defined by the reward signal (the goal) plus the environment dynamics (states, actions, transitions) — without a reward there is no RL problem. The *solution* is the policy (and the value function used to find it); the model is optional, used only by model-based solutions. So reward + environment define the problem; policy/value define the solution.

Q4 · Review

Identify a real problem solvable by RL and define its RL elements.

✓ Solution

Example: a thermostat/HVAC controller. **State** = current temperature, occupancy, time, outdoor weather; **actions** = heat/cool/idle; **reward** = comfort minus energy cost; **policy** = chosen action per state; **value** = expected long-run comfort-minus-cost; **model** (optional) = a thermal model of the building. The agent learns to trade comfort against energy over time.

Q5 · Review

How is RL different from supervised and unsupervised learning?

✓ Solution

Supervised learning trains on labelled (input, correct-output) pairs; RL has no labelled correct actions, only evaluative reward, often delayed. Unsupervised learning finds hidden structure in unlabelled data; RL's goal is not structure but maximising cumulative reward through interaction. RL's distinctive features are trial-and-error search and delayed reward, with the agent's own actions affecting the data it sees.

Q6 · Review

“A reward signal defines the goal of a reinforcement learning problem.” Explain.

✓ Solution

The reward encodes *what* we want, not *how* to achieve it: by maximising expected cumulative reward the agent is driven toward the designer's goal. Designing the reward is therefore how we specify the task; a poorly shaped reward makes the agent optimise the wrong thing (reward hacking). The agent never gets told the right actions — only how good outcomes are.

Q7 · Review

What is a model of the environment, and how does the solution differ with vs without one?

✓ Solution

A model predicts the environment's response — next state and reward — given a state and action. **With** a model (model-based RL) the agent can *plan*, e.g. dynamic programming, which is sample-efficient but needs an accurate model. **Without** a model (model-free RL, e.g. Monte Carlo, TD) the agent learns values/policies directly from experience — simpler and robust to model error, but needing more interaction.

Q8 · Review

Explain the exploration-exploitation trade-off and why it matters.

✓ Solution

Exploitation uses the best action known so far to earn reward now; exploration tries uncertain actions to gather information that may yield more reward later. It matters because pure exploitation can lock onto a sub-optimal action (never discovering better ones), while pure exploration wastes reward. Good performance requires balancing the two.

Q9 · Review

What happens to an agent that constantly explores, or constantly exploits? Are such agents ever useful?

✓ Solution

A constant explorer never settles on good behaviour — it gathers information but accrues low reward; a constant exploiter freezes on its current (possibly wrong) best and never improves. Each has niche value: pure exploration is useful for an initial data-collection / environment-mapping phase, and pure exploitation is appropriate once the values are known to be accurate (e.g. at deployment with a fixed, well-learned policy).

Q10 · Review

What are different ways of choosing ϵ in ϵ -greedy, and how does each affect the trade-off?

✓ Solution

Fixed ϵ (e.g. 0.1): constant exploration forever — simple, good for non-stationary problems. **Decaying** ϵ (e.g. $\epsilon_t = 1/t$ or exponential decay): explore heavily early, exploit more as estimates improve — good for stationary problems. **Larger** ϵ explores more (more robust but lower immediate reward); **smaller** ϵ exploits more. The choice scales with reward uncertainty/variance.

Q11 · Review

Tic-Tac-Toe: why update $V(S_t)$ only after *greedy* (exploitative) moves and not after exploratory moves? Sketch ϵ -greedy pseudo-code.

✓ Solution

We back up values to estimate the value of *following the current (greedy) policy*; an exploratory move is a deliberate deviation, so the outcome after it does not reflect the policy's value and would bias $V(S_t)$ toward the random choice. Updating only after greedy moves keeps the estimates consistent with the policy being improved. Pseudo-code per step: observe state S ; with prob ϵ pick a random legal move (exploration, *no* value update); else pick $\arg \max$ over next states of V (greedy) and update $V(S) \leftarrow V(S) + \alpha[V(S') - V(S)]$; play, repeat until the game ends.

Review: Module 2 — Multi-Armed Bandits

Q12 · Review

Explain why RL feedback is evaluative, not instructive, with an example.

✓ Solution

Instructive feedback tells you the *correct* action regardless of what you did (as in supervised learning); evaluative feedback only tells you how *good* the action you took was, not what the best action would have been. Example: in a k -armed bandit, pulling an arm returns a reward sample for *that* arm only — you never learn what the other arms would have paid, so you must explore to evaluate them.

Q13 · Review

Why is a multi-armed bandit an RL problem in a *non-associative* setting?

✓ Solution

It has the core RL feature — learning to act from evaluative reward — but there is a single situation (no distinct states to associate actions with), so the agent need not learn *which* action suits *which* state. It is ‘non-associative’ because the best action is the same every step; adding states (associating actions with situations) turns it into a full (associative) RL problem.

Q14 · Review

Give one application modelled as a multi-armed bandit and why it fits.

✓ Solution

Online ad / article selection (or A/B testing): each candidate ad is an arm, the reward is a click/conversion, and there is essentially one context repeated many times. It fits because we must repeatedly choose among options with unknown payoffs and balance trying new ads (exploration) against showing the best-known ad (exploitation) to maximise total clicks.

Q15 · Review

If the action value is constant (the same) for all actions, do you need to explore? Why or why not?

✓ Solution

No. If every action has the same true value, no action is better than another, so exploration cannot improve the expected reward — any choice is optimal. Exploration only pays off when there are differences between actions to discover; with identical values there is nothing to learn.

Q16 · Review

Tabulate the approaches to balance exploration and exploitation, with a remark on each.

✓ Solution

ϵ -greedy — explore uniformly with prob ϵ ; simple, but explores indiscriminately. **Decaying ϵ** — explore less over time; good for stationary problems. **Optimistic initial values** — high initial estimates force early trials; temporary, stationary only. **UCB** — explore by uncertainty ($Q + c\sqrt{\ln t/N}$); directed, efficient. **Gradient bandit (softmax)** — probabilistic by preference; smooth. Each trades simplicity against how cleverly it targets exploration.

Q17 · Review

What ϵ would you choose for (a) deterministic rewards, (b) low-variance rewards, (c) high-variance rewards? Why?

✓ Solution

(a) Deterministic: $\epsilon \approx 0$ — one sample reveals an action’s true value, so almost no exploration is needed. (b) Low variance: small ϵ — few samples give reliable estimates. (c)

High variance: larger ϵ — noisy rewards need many samples per action to estimate the mean, so more exploration is justified. Exploration should scale with reward uncertainty.

Q18 · Review

In ϵ -greedy with four actions (a,b,c,d; a is greedy) and $\epsilon = 0.4$, find the probability the greedy and each non-greedy action is selected.

✓ Solution

With prob $1 - \epsilon$ we take the greedy action, and with prob ϵ we pick uniformly among all $k = 4$ actions. So $P(\text{greedy } a) = (1 - \epsilon) + \epsilon/k = 0.6 + 0.4/4 = 0.6 + 0.1 = \mathbf{0.7}$, and each non-greedy (b, c, d) has $P = \epsilon/k = 0.4/4 = \mathbf{0.1}$. Check: $0.7 + 3(0.1) = 1.0$.

Q19 · Review

For the update $\text{NewEstimate} = \text{OldEstimate} + \text{StepSize}[\text{Target} - \text{OldEstimate}]$, comment on (a) step size $1/n$ and (b) constant α for non-stationarity and high variance.

✓ Solution

(a) $\alpha_n = 1/n$ gives the *sample average*: all rewards weighted equally, converging to the true mean — ideal for *stationary* problems and it averages out high variance, but it cannot track a changing (non-stationary) value because old data dominates. (b) Constant α gives an *exponentially recency-weighted* average: it keeps tracking a drifting value (good for non-stationarity) but never fully averages out noise, so under high variance the estimate stays jittery.

Q20 · Review

Why is the exploration from optimistic initial values temporary, and when would you use it?

✓ Solution

Setting initial estimates far above realistic rewards makes every tried action ‘disappoint’ (its estimate drops below the still-optimistic untried ones), forcing the agent to try them all early. Once all actions have been sampled a few times the estimates settle near their true values and the optimism — and hence the forced exploration — fades. Use it as a simple exploration hack for *stationary* problems; it is poorly suited to non-stationary ones.

Q21 · Review

How do policy-based and value-based methods differ?

✓ Solution

Value-based methods learn a value function (e.g. Q) and derive the policy greedily from it ($\arg \max_a Q$); they suit discrete action sets. Policy-based methods parametrise and optimise the policy directly (e.g. by gradient ascent on expected reward), handling continuous/stochastic actions naturally and learning stochastic policies. Actor-critic methods combine both.

Q22 · Review

Name two approaches to maintain the estimate of action values.

✓ Solution

(1) **Sample-average** incremental update $Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n)$ — equal weighting, converges to the mean. (2) **Constant step-size** update $Q \leftarrow Q + \alpha(R - Q)$ — exponentially recency-weighted, tracks non-stationary values. (Both avoid storing all past rewards.)

Q23 · Review

How does UCB differ from ϵ -greedy?

✓ Solution

ϵ -greedy explores *uniformly at random*, ignoring how uncertain each action's estimate is. UCB explores *by uncertainty*: it picks $\arg \max_a [Q_t(a) + c\sqrt{\ln t / N_t(a)}]$, adding a bonus that is large for rarely-tried actions and shrinks as an action is sampled. So UCB targets exploration where it is most informative, usually achieving lower regret than ϵ -greedy.

Q24 · Review

What factors determine the right ϵ for a given problem?

✓ Solution

Reward variance/noise (more noise $\Rightarrow$ larger ϵ), stationarity (non-stationary $\Rightarrow$ keep $\epsilon > 0$, often constant), the number of actions, the time horizon (long horizon allows more early exploration), and the cost of a mistake. Often ϵ is decayed over time to explore early and exploit later.